

IA DONINHA

Middleware Neuro-Simbólico Híbrido para Raciocínio Epistemológico em Modelos de Linguagem

Autor: Daniel Barros Fonseca

Contato: danielfonseca420@proton.me

Patente em processo de registro — INPI (Brasil)

Resumo

Este artigo apresenta a IA Doninha, um middleware de software neuro-simbólico híbrido interposto entre aplicações cliente e Modelos de Linguagem de Grande Escala (LLMs) externos. O sistema opera como proxy epistêmico transparente, interceptando chamadas bidirecionalmente, submetendo o prompt original a um pipeline de raciocínio estruturado em sete camadas funcionais (L1–L7) e devolvendo uma resposta enriquecida com metadados de certeza, contradição e valor-verdade auditáveis.

O pipeline combina extração semântica aristotélica (L1), classificação kantiana de proposições (L2), filtragem científica por critérios de Hempel e Popper (L2.5), avaliação paraconsistente baseada na Lógica Anotada de Evidências de Newton da Costa (L3), síntese por equivalência russelliana com Chain of Verification (L4), geração textual pelo LLM base (L5), refinamento de fluência (L6) e entrega de resposta auditável com rastro de cadeia de pensamento hierárquico (L7). Diferentemente das soluções existentes de prompt engineering e guardrails de pós-processamento, a IA Doninha atua no pré-processamento estruturado do prompt e no pós-processamento epistemológico da resposta, sem exigir modificação do cliente nem do LLM externo.

1. Introdução

1.1. O Problema Estrutural dos LLMs Atuais

Os Modelos de Linguagem de Grande Escala disponíveis comercialmente — incluindo modelos baseados em arquitetura Transformer como GPT (OpenAI), Claude (Anthropic), Gemini (Google) e equivalentes — são acessados por aplicações cliente mediante chamadas a APIs padronizadas, nas quais o cliente envia um prompt em linguagem natural e recebe uma resposta gerada pelo modelo. Esse paradigma, embora amplamente adotado, apresenta limitações estruturais documentadas que comprometem a confiabilidade e a auditabilidade das respostas em domínios de alta exigência epistêmica.

As principais limitações são:

- Ausência de pré-processamento epistêmico: o prompt chega ao LLM exatamente como formulado pelo usuário, sem decomposição formal em conceitos, hipóteses ou juízos.
- Inexistência de quantificação de incerteza na entrada: o LLM não recebe informações sobre grau de certeza, contradição ou indeterminação das proposições do prompt.
- Ausência de verificação de consistência pré-geração: contradições lógicas presentes no prompt são transmitidas integralmente ao modelo.
- Opacidade do raciocínio intermediário: o processo de transformação entre prompt e resposta ocorre inteiramente no interior do LLM, sem etapa auditável acessível ao cliente.
- Dependência de engenharia de prompt manual: a qualidade da resposta depende da habilidade do usuário em formular o prompt adequado.

- Incapacidade de tratar valores-verdade intermediários: a lógica binária clássica dos LLMs colapsa ante proposições que requerem valores não absolutos — fenômeno conhecido como alucinação de máquina.

Soluções anteriores — como sistemas de prompt engineering automatizado, guardrails de saída e wrappers de API — atuam predominantemente no pós-processamento da resposta ou na filtragem superficial do prompt, sem implementar raciocínio epistemológico estruturado na camada de interceptação bidirecional.

1.2. A Proposta da IA Doninha

A IA Doninha endereça essas limitações por meio de uma abordagem original: interpor, de forma transparente, uma camada de middleware que aplica raciocínio neuro-simbólico híbrido a cada chamada antes de encaminhá-la ao LLM externo. O sistema combina:

- Geração estatística de linguagem natural pelo LLM base (componente neural);
- Estruturação simbólica das proposições mediante categorias aristotélicas, juízos kantianos e silogismo científico;
- Verificação epistemológica por lógica paraconsistente anotada, síntese russelliana e Chain of Verification.

Definição técnica:

Um middleware neuro-simbólico híbrido é um sistema que combina redes neurais — responsáveis pela geração estatística de linguagem — com mecanismos simbólicos baseados em regras lógicas e estruturas de conhecimento formal. Ao contrário dos modelos puramente neurais (que calculam probabilidades de tokens) ou puramente simbólicos (que operam exclusivamente por regras explícitas), o sistema híbrido aproveita os pontos fortes de ambas as abordagens.

2. Arquitetura Geral

2.1. Posicionamento como Proxy Epistêmico Transparente

O middleware posiciona-se entre a aplicação cliente e o LLM externo, operando em dois planos de interceptação independentes e complementares:

Plano	Função
Upstream (entrada)	Intercepta o prompt antes de sua transmissão ao LLM externo. O prompt original é processado pelo pipeline L1–L7 e substituído por um prompt enriquecido com proposições hierarquizadas, graus de verdade quantificados e instruções de calibração epistêmica.
Downstream (saída)	Intercepta a resposta gerada pelo LLM externo antes de sua entrega ao cliente. A resposta é anotada com metadados epistêmicos — certeza (Gc), contradição (Gct), estado paraconsistente, valor-verdade (v) e log de verificação CoVe — e devolvida no formato original da API.

A transparência é garantida por dois mecanismos: (i) o middleware implementa a mesma interface de entrada e saída do LLM externo alvo, de modo que a aplicação cliente não requer alteração de código; (ii) o middleware encaminha as chamadas ao LLM externo utilizando as credenciais e endpoints configurados, atuando como proxy epistêmico invisível.

2.2. Visão do Fluxo Completo

O fluxo de processamento de cada chamada interceptada segue a sequência abaixo:

Etapa	Módulo Principal	Função Resumida
L1	ConceptTable + LogicLMSymbolicSolver	Extração semântica e mapeamento aristotélico de termos do prompt
L2	KantianJudgmentEngine + BERTAssertionClassifier	Classificação e priorização epistêmica por tábua dos doze juízos de Kant
L2.5 ★	ScientificSyllogismPipeline	Filtragem científica: validação silogística, critério de Hempel e falsificabilidade popperiana
L3	ParaconsistentEngine + Weighted Dynamic Ensemble	Atribuição de μ , λ , certeza (Gc), contradição (Gct), valor-verdade (v) e estado lógico
L4	RussellianSynthesisEngine + ChainOfVerificationAgent	Síntese por equivalência proposição-mundo e verificação via CoVe
L5	generate_response + providers	Geração de resposta textual em linguagem natural com contexto L1–L4
L6	FinalResponseEngine	Polimento de clareza, coesão e tom; preservação dos dados epistêmicos
L7	FinalTextEngine + HierarchicalCoTTrace	Entrega auditável com blocos [AUDIT] e rastro de cadeia de pensamento por camada

★ Etapa presente na versão atual e ausente em versões anteriores

3. Pipeline L1–L7: Descrição Detalhada

3.1. L1 — Tábua de Conceitos

O Problema que L1 Resolve

Os LLMs puramente neurais recebem o prompt como uma sequência de tokens e calculam probabilisticamente qual deve ser o próximo. Esse processo é cego a distinções semânticas fundamentais: sinônimos, antônimos, termos ambíguos e conceitos contextualmente incompatíveis são tratados do mesmo modo. O resultado são respostas que refletem diretamente os vieses e ambiguidades da entrada.

Kant escreveu que "pensamentos sem conteúdo são vazios; intuições sem conceitos são cegas". L1 existe para fornecer conteúdo ao sistema: extrai os conceitos essenciais do prompt, mapeia as relações entre eles e entrega essa estrutura semântica como base para todas as camadas subsequentes.

Funcionamento

O módulo principal de L1 é o ConceptTable, que opera em quatro etapas sequenciais:

- Extração de conceitos do prompt com filtragem de ruído;
- Mapeamento de relações aristotélicas binárias: sinonímia, antonímia, hiperonímia (termo mais geral), hiponímia (termo mais específico), homonímia e paronímia;
- Consulta à base de conhecimento local (KB) e, quando disponível, ao contexto RAG via injeção de embeddings vetoriais;
- Filtragem de conceitos contextualmente incompatíveis pelo solver simbólico LogicLM, que verifica compatibilidade canônica antes de passar o material para L2.

O SpacySemanticTermExtractor é um componente opcional que permite identificação de sintagmas complexos e expressões multipalavras via spaCy — útil para domínios técnicos com terminologia especializada. O LogicLMSymbolicSolver complementa a extração verificando se os conceitos identificados são compatíveis com o contexto do prompt, gerando alertas canônicos quando detecta incongruências.

Por que isso importa:

Grande parte das alucinações de máquina deriva de erros do LLM na identificação do contexto do prompt. Ao construir uma tábua de conceitos explícita antes da geração, L1 reduz o espaço de ambiguidade semântica que o modelo precisa resolver internamente durante a predição de tokens.

3.2. L2 — Juízos Kantianos

O Problema que L2 Resolve

L1 entrega um conjunto de conceitos semânticamente organizados. L2 transforma esse material em proposições estruturadas segundo critérios lógicos formais, atribuindo prioridade epistêmica

a cada uma. Sem essa estruturação, o motor paraconsistente de L3 receberia conceitos brutos sem hierarquia — o equivalente a tentar calcular probabilidades sobre texto não estruturado.

A Tábua dos Doze Juízos

O KantianJudgmentEngine classifica cada proposição extraída do prompt segundo as quatro dimensões da tábua kantiana dos juízos, que estrutura a faculdade do entendimento humano segundo categorias formais do pensamento:

Dimensão	Categorias
Quantidade	Universal ("todo"), Particular ("algum"), Singular ("este")
Qualidade	Afirmativo, Negativo, Infinito
Relação	Categórico ("S é P"), Hipotético ("se... então"), Disjuntivo ("ou... ou")
Modalidade	Problemático (possível), Assertório (real), Apodítico (necessário)

Para cada proposição extraída, o motor atribui um grau de prioridade epistêmica (forte, intermediária, fraca) e classifica o domínio da hipótese em quatro categorias operacionais: empiricamente verificado, teoricamente plausível, logicamente contraditório ou empiricamente indeterminado. Esse domínio funciona como prior bayesiano para o cálculo de μ e λ em L3.

O BERTAssertionClassifier, quando disponível, complementa a classificação kantiana com uma análise bidirecional baseada em Transformers, classificando cada proposição em verdadeiro (T), indeterminado (I) ou falso (F) conforme evidências extraídas do contexto. A integração entre classificação kantiana e BERT bidirecional — mediada pela estrutura formal da tábua dos juízos — é um dos diferenciais técnicos da arquitetura.

3.3. L2.5 — Silogismo Científico (Etapa Intermediária)

Por que Esta Etapa Existe

L2 pode gerar hipóteses kantianamente bem-formadas que apresentam dois problemas distintos: são semanticamente distantes do prompt original (irrelevantes, no sentido de Hempel) ou são proposições que não poderiam ser refutadas por nenhuma evidência possível (não falsificáveis, no sentido de Popper). Essas hipóteses, se encaminhadas diretamente a L3, introduzem ruído no cálculo das métricas paraconsistentes. A etapa intermediária filtra esse ruído antes de L3.

Os Três Filtros

- **AristotelianSyllogismValidator:** verifica se as relações lógicas entre premissas e conclusão são silogisticamente válidas. Hipóteses com inferências inválidas são marcadas e removidas.
- **HempelFilter:** aplica o critério de relevância de Carl Hempel, descartando hipóteses com baixa relação semântica com o prompt original. Evita que o motor paraconsistente processe proposições sobre tópicos não pertinentes ao que o usuário efetivamente perguntou.

- **PopperFalsifiability:** avalia se cada hipótese é empiricamente testável — isto é, se poderia ser refutada por alguma evidência possível. Hipóteses vagas demais (ex.: afirmações absolutas imunes a qualquer contra-exemplo) são removidas antes de chegar ao motor paraconsistente.

O ScientificSyllogismPipeline orquestra os três filtros em sequência e entrega a L3 apenas as hipóteses que passaram por todos os critérios. Isso resulta em um conjunto menor e mais preciso de proposições para avaliação, o que melhora a qualidade dos estados lógicos produzidos por L3.

3.4. L3 — Motor Paraconsistente

O Problema das Explosões Lógicas

A lógica clássica binária — adotada por todos os LLMs comerciais atuais — sofre de uma limitação fundamental: quando o sistema encontra uma contradição (proposição P e sua negação $\neg P$ ambas presentes), qualquer proposição pode ser provada como verdadeira. Esse fenômeno, chamado de trivialização ou explosão lógica, está na raiz das alucinações de máquina: o modelo, incapaz de lidar formalmente com a contradição, produz uma resposta aleatória ou evasiva que não reflete nenhuma evidência real.

Newton da Costa, matemático brasileiro, desenvolveu a Lógica Paraconsistente Anotada de Evidências (LAE) precisamente para resolver esse problema: sistemas paraconsistentes permitem que contradições locais existam sem que o sistema como um todo colapse. Da Costa chamou isso de "explosão gentil": contradições locais não geram contradições globais.

O Motor Paraconsistente: Métricas e Estados

O ParaconsistentEngine avalia cada proposição entregue por L2.5 calculando cinco métricas a partir de dois valores de entrada:

Métrica	Fórmula	Significado
μ (mu)	Score de similaridade semântica favorável	Grau de evidência favorável à proposição $\in [0, 1]$
λ (lambda)	Score de similaridade semântica contrária	Grau de evidência contrária à proposição $\in [0, 1]$
Gc	$Gc = \mu - \lambda$	Grau de certeza $\in [-1, +1]$
Gct	$Gct = \mu + \lambda - 1$	Grau de contradição $\in [-1, +1]$
v	$v = (\mu - \lambda + 1) / 2$	Valor-verdade normalizado $\in [0, 1]$

Com base nos valores de Gc e Gct , o para-analisador classifica cada proposição em um dos doze estados lógicos do reticulado QUPC (Quadrado Unitário do Plano Cartesiano). Os quatro estados extremos correspondem aos vértices do quadrado:

- $D = (1, 0)$: Verdadeiro — certeza máxima afirmativa, sem contradição;
- $B = (0, 1)$: Falso — certeza máxima negativa, sem contradição;
- $C = (1, 1)$: Inconsistente — alta contradição positiva;
- $A = (0, 0)$: Indeterminado — ausência de evidências em ambos os sentidos.

Entre esses extremos existem oito estados intermediários (Quase-Verdadeiro, Quase-Falso, Inconsistente_to_V, QV_to_Inconsistente, Indeterminado_to_V, Indeterminado_to_F, QV_to_Indeterminado e QF_to_V) que permitem ao sistema expressar nuances epistêmicas impossíveis na lógica binária clássica.

Weighted Dynamic Ensemble: A Dimensão Neural de L3

A versão atual de L3 combina duas fontes de cálculo de μ e λ por meio de um ensemble com pesos dinâmicos:

- **Fonte heurística:** scoring de similaridade semântica entre a proposição e os documentos da base de conhecimento, ajustado pelos priors epistêmicos entregues por L2 (domínio e prioridade da hipótese).
- **Fonte neural (TruthScoringModel):** classificador Transformer treinado especificamente para produzir estado paraconsistente e valor-verdade a partir de proposições em linguagem natural.

Os pesos do ensemble são calculados dinamicamente conforme a concordância entre as duas fontes: o peso heurístico equivale a 0,65 somado a 0,25 vezes o grau de concordância entre as fontes, enquanto o peso neural corresponde ao complemento do peso heurístico em relação a 1,0.

Quando as duas fontes concordam (alto agreement), o peso heurístico sobe para até 0.90. Quando divergem, a proporção se aproxima de 65/35. O sistema registra `ensemble_agreement`, `neural_state` e `neural_truth` como metadados nos blocos de auditoria. Quando o TruthScoringModel não está disponível, L3 opera em modo heurístico puro com `heuristic_weight = 1.0`.

Dois mecanismos de controle derivam do estado paraconsistente:

(1) Regra de busca de evidências adicionais: quando $G_{ct} \geq +0.5$, o pipeline instrui L1 a ampliar a consulta à KB ou acionar o agente de busca web antes de prosseguir; (2) Regra de emissão de conclusão: quando $G_{ct} < +0.5$ e $|G_c| \geq +0.5$, o sistema dispõe de certeza suficiente e a conclusão é emitida para L4 com metadados de qualidade.

3.5. L4 — Síntese Russelliana com Chain of Verification

Fundamento Teórico

L4 é o núcleo da dimensão verificacional do sistema. Ela parte do princípio estabelecido por Bertrand Russell em *The Problems of Philosophy*: a verdade de uma proposição não é uma

predição estatística sobre qual palavra deve vir a seguir, mas um grau de equivalência entre a proposição e os fatos representados em uma base de conhecimento. L4 implementa esse princípio computacionalmente.

Funcionamento

L4 opera em três módulos complementares:

- **RussellConceptBase:** extrai sentenças sobre correspondência, crença, fato e verdade do texto de Russell (data/russell.txt), gera uma tabela de pesos conceituais normalizada entre 0,2 e 1,0, e calcula um score de equivalência russelliana para cada proposição.
- **RussellianSynthesisEngine:** recebe as hipóteses paraconsistentes de L3, as prioridades kantianas de L2 e o prompt original. Seleciona a hipótese de maior valor-verdade como base da resposta e calcula um valor-verdade final ponderado combinando prioridade kantiana (L2), certeza paraconsistente (Gc) e score de equivalência russelliana na proporção 0,7/0,3.
- **ChainOfVerificationAgent:** aplica o padrão Chain of Verification (CoVe) na variante Factor + Revise. Gera entre seis e doze perguntas de verificação independentes sobre a resposta sintetizada, classifica cada uma como Confirmado, Refutado, Parcialmente correto ou Incerto, e reescreve a resposta incorporando apenas o que foi efetivamente verificado.

O resultado é encapsulado na estrutura `SynthesisResult`, que reúne a resposta revisada, o valor-verdade agregado, a certeza (Gc), o grau de contradição (Gct), o estado lógico, as evidências de suporte, as hipóteses falsificadas, o log de verificação e um rótulo de confiança derivado automaticamente (de "Indeterminado" a "Alta Confiança").

3.6. L5, L6 e L7 — Geração, Refinamento e Síntese Final

L5 — Geração Textual

L5 transforma a síntese estruturada de L4 em linguagem natural fluente. O diferencial em relação a um LLM comum é que o modelo não parte do zero: recebe contexto estruturado com conceitos (L1), proposições priorizadas (L2), estados paraconsistentes (L3) e síntese verificada (L4), o que reduz significativamente o espaço de alucinações possíveis na geração.

O módulo `generate_response` roteia a geração para o provider configurado (Ollama, OpenAI, Anthropic, Gemini, Grok, Groq, Meta/Llama, `custom_lm` local ou `fallback template`), garantindo operação mesmo sem provider externo disponível.

L6 — Refinamento Final

L6 realiza o polimento do texto gerado em L5: melhora clareza, fluidez e coesão; ajusta o tom ao grau de confiança calculado; menciona explicitamente incertezas e contradições quando presentes; e assegura consistência terminológica com a tábua de conceitos de L1. O `EpistemicContext` agrega os estados de L3, as rotas paraconsistentes e as classificações BERT para que o refinamento seja informado pelo contexto epistêmico completo.

L7 — Síntese Definitiva Auditável

L7 integra os resultados de L1–L6 na resposta final entregue ao cliente. A camada classifica o perfil da audiência (leigo, técnico ou acadêmico) e ajusta o tom e o nível de detalhe conforme esse perfil. Mais importante: L7 acrescenta blocos de auditoria ([AUDIT L4], [AUDIT L5], [AUDIT L6], [AUDIT L7]) que registram provider utilizado, modelo, valor-verdade, certeza, contradição, estado lógico, fontes bibliográficas e, quando solicitado, o rastro completo de Chain of Thought hierárquico por camada (HierarchicalCoTTrace).

Uso responsável em domínios sensíveis:

A presença dos blocos [AUDIT] transforma a resposta de uma caixa preta em um documento rastreável. Em áreas como direito, medicina, educação e ciência, onde a fonte e o grau de certeza de uma afirmação são juridicamente e metodologicamente relevantes, essa auditabilidade é um diferencial operacional concreto, não apenas uma característica arquitetural.

4. Componentes de Suporte

4.1. RAG Híbrido e Base de Conhecimento

O sistema suporta quatro estratégias de recuperação de contexto, configuráveis por domínio:

- **direct_injection**: injeção direta de documentos da KB no contexto do prompt — mais rápida e determinística.
- **semantic_retrieval**: busca por similaridade vetorial em ChromaDB usando HuggingFace Embeddings — recupera os documentos mais semanticamente próximos ao prompt.
- **hybrid**: combina as duas estratégias anteriores, balanceando velocidade e cobertura semântica.
- **domain_aware**: detecta automaticamente o domínio do prompt (medicina, direito, ciência etc.) e seleciona a KB específica do domínio antes de recuperar documentos.

A Knowledge Base pode ser carregada de arquivos JSON, JSONL ou documentos de texto, enriquecida via ChromaDB, mesclada entre fontes e acessada por domínio específico. A base mínima de fallback (SEED_KNOWLEDGE_BASE) garante operação mesmo sem KB configurada.

4.2. Agente de Busca Web

O agente implementado em `agente_busca_web.py` combina busca local em ChromaDB com busca externa via DuckDuckGo em um agente ReAct (Reasoning + Acting) construído sobre LangChain. Sua ativação não é genérica — é condicionada ao estado epistêmico produzido por L3:

- Estado Indeterminado ($Gct \leq -0.5$): ausência de evidências em ambos os sentidos → agente ativado para buscar contexto externo;
- Fallback heurístico ativo (TruthScoringModel indisponível) → agente ativado como enriquecimento compensatório;
- Estado Inconsistente ($Gct \geq +0.5$): alta contradição → agente ativado para buscar evidências que permitam resolver a contradição.

Essa ativação condicional transforma a busca web de um enriquecimento genérico do prompt em um mecanismo de resolução específico de estados epistêmicos problemáticos.

4.3. Modelo de Linguagem Customizável

Para implantações que exigem operação totalmente local — sem dependência de nenhum provider externo — o sistema inclui infraestrutura completa para treinar um modelo de linguagem próprio: tokenizador SentencePiece (`custom_tokenizer.py`), arquitetura Transformer customizada (`custom_lm_model.py`) e rotinas de pré-treinamento em corpus filosófico local (`pretrain_custom_lm.py`, `run_pretrain.py`). Quando configurado como provider `custom_lm`, o sistema opera inteiramente on-premise.

4.4. Chain of Thought Auditável

O HierarchicalCoTTrace registra um passo por camada do pipeline (CoTStep), contendo camada, título, resumo do raciocínio, decisões-chave, saída e duração. O rastro completo pode ser exportado como dicionário estruturado ou como Markdown legível, e é acessível via `return_cot=True` no pipeline. Isso permite que integrações cliente inspecionem o raciocínio intermediário do sistema sem acesso ao código-fonte.

5. Interfaces e Providers

5.1. Interfaces Disponíveis

Interface	Arquivo	Uso
CLI	pipeline.py	Linha de comando: --prompt, --demo, --repl
API REST	api.py	FastAPI com endpoints /health, /process, /chat, /agent
Chat UI	app.py	Interface Chainlit com streaming via Ollama
Standalone	doninha_standalone.py	Arquivo consolidado para distribuição sem dependências separadas

5.2. Providers de LLM Suportados

O módulo `llm_provider_client.py` normaliza o acesso a nove providers de LLM externos, mais dois modos locais:

Provider	Tipo
Ollama	Local — modelo doninha8:latest (padrão recomendado)
OpenAI	Remoto — compatível com API OpenAI
Anthropic (Claude)	Remoto — via ANTHROPIC_API_KEY
Google Gemini	Remoto — via GEMINI_API_KEY
xAI Grok	Remoto — via GROK_API_KEY
Groq	Remoto — inferência acelerada, compatível com OpenAI
Meta / Llama API	Remoto — acesso a modelos Llama via API Meta
custom_lm	Local — modelo treinado na própria infraestrutura
template	Fallback local sem chamada externa — garante operação offline total

6. Fundamentos Teóricos

A IA Doninha é a prova de conceito implementada das proposições desenvolvidas pelo criador no artigo "Uma verdadeira Epistemologia para a Inteligência Artificial". Os principais referenciais filosóficos e científicos que fundamentam a arquitetura são:

Referencial	Camada	Contribuição ao Sistema
Aristóteles (Tópicos / Categorias)	L1, L2.5	Relações semânticas fundamentais (sinonímia, antonímia, hipo/hiperonímia) e validação silogística
Immanuel Kant (Crítica da Faculdade do Juízo)	L2	Tábua dos doze juízos como estrutura formal de classificação de proposições; distinção analítico/sintético, a priori/a posteriori
Newton da Costa (LAE)	L3	Lógica Paraconsistente Anotada de Evidências; tratamento formal de contradições sem trivialização do sistema
Bertrand Russell (Problems of Philosophy)	L4	Teoria da correspondência e equivalência como princípio de alinhamento proposição-mundo real
Carl Hempel (Critério de Relevância)	L2.5	Filtragem de hipóteses semanticamente irrelevantes para o prompt
Karl Popper (Falsificabilidade)	L2.5	Critério de demarcação científica: eliminação de proposições não falsificáveis
Antoine Culioli (Linguística Enunciativa)	L2	Análise enunciativa integrada à classificação BERT para contextualização semântica dos juízos

7. Diferenciais Técnicos e Inovação

A seguir, os aspectos que distinguem a IA Doninha das soluções existentes de integração com LLMs:

- **Primeiro middleware transparente de interceptação bidirecional** que implementa raciocínio epistemológico estruturado em múltiplas camadas sobre chamadas a LLMs externos, sem modificação do cliente ou do LLM.
- **Primeira aplicação combinada, em sistema operacional de software**, de: (a) tábua de conceitos aristotélica; (b) tábua de juízos kantianos; (c) lógica paraconsistente com métricas $\mu/\lambda/Gc/Gct/v$; (d) síntese por equivalência russelliana; e (e) Chain of Verification — como pipeline sequencial de enriquecimento de prompt para LLMs.
- **Resolução técnica original do problema das explosões lógicas** em sistemas de linguagem, mediante substituição do paradigma de lógica binária clássica por avaliação paraconsistente que preserva valores-verdade intermediários.
- **Geração de rastro de auditoria epistêmica integrado ao fluxo de resposta**, permitindo verificabilidade do raciocínio intermediário de forma acessível à aplicação cliente — funcionalidade inexistente nos sistemas anteriores.
- **Arquitetura modular** que permite substituição ou atualização independente de qualquer camada (L1–L7) sem impacto nas demais, viabilizando evolução incremental e adaptação a diferentes LLMs base.
- **Suporte nativo a RAG híbrido** integrado ao pipeline epistemológico, com estratégias configuráveis de combinação entre recuperação vetorial local e busca externa.
- **Filtragem científica pré-paraconsistente** (Hempel + Popper) que elimina hipóteses irrelevantes e não falsificáveis antes do cálculo das métricas epistêmicas — reduzindo ruído de entrada em L3.
- **Weighted Dynamic Ensemble em L3** que combina heurística simbólica com modelo neural treinável (TruthScoringModel) por pesos calculados dinamicamente a partir da concordância entre as duas fontes.

A fundamentação teórica desta abordagem e a proposta de sua superação por modelo híbrido foram desenvolvidas pelo criador e expostas no artigo "Uma verdadeira Epistemologia para a Inteligência Artificial" (Daniel Barros Fonseca), antes da implementação do sistema. A IA Doninha constitui a prova de conceito implementada dessas proposições teóricas originais.

8. Avaliação e Métricas

A versão atual inclui infraestrutura de avaliação sistemática, composta por métricas objetivas e suites de testes por componente:

Métrica / Teste	O que avalia
coherence_l3	Coerência entre valor-verdade, estado lógico e contradição produzidos por L3
BLEU	Sobreposição de n-gramas entre resposta gerada e referência
ROUGE-L	Maior subsequência comum entre resposta gerada e referência
Similaridade semântica	Distância vetorial via SentenceTransformers entre resposta e referência
test_epistemic_classification	Corretude da classificação epistêmica em L2 e L3
test_l7_tone_guidance	Adequação do tom e nível de detalhe para cada perfil de audiência em L7
test_citation_behavior	Presença e formato correto de fontes nos blocos [AUDIT]
test_rag_hybrid	Qualidade e relevância do contexto recuperado pelas estratégias RAG

9. Configuração e Execução

9.1. Configuração Mínima

O arquivo `config.yaml` centraliza todos os parâmetros do pipeline. Configuração mínima para operação local:

Na seção de geração, define-se o provedor como Ollama, com o modelo `doninha8:latest` e o endereço local `http://localhost:11434`. Na seção de finalização, o mesmo provedor e modelo são repetidos. Na seção referente a L7, configura-se igualmente o provedor Ollama com o modelo `doninha8:latest`.

9.2. Execução Básica

Após instalar as dependências e iniciar o Ollama:

- Processar um prompt único pela linha de comando, informando o texto desejado diretamente como parâmetro de execução;
- Iniciar o modo interativo (REPL), que mantém uma sessão contínua de perguntas e respostas;
- Iniciar a API REST, disponibilizando os endpoints do sistema para integração com outras aplicações.

9.3. Exemplo de Saída com Auditoria

Uma resposta típica do sistema inclui o texto gerado em linguagem natural pelas camadas L5 a L7, seguido dos blocos de auditoria com os seguintes metadados:

Bloco de Auditoria	Metadados Registrados
[AUDIT L4]	Provedor e modelo utilizados (ollama / doninha8:latest); estado lógico (ex.: Verdadeiro); valor-verdade (ex.: 0,87); grau de certeza Gc (ex.: +0,74); grau de contradição Gct (ex.: 0,02); fontes consultadas (ex.: data/russell.txt, base de conhecimento local)
[AUDIT L7]	Perfil de audiência identificado (ex.: técnico); grau de concordância do ensemble (ex.: 0,82); estado neural (ex.: Verdadeiro); rótulo de confiança (ex.: Alta Confiança)

10. Conclusão

A IA Doninha representa uma abordagem fundamentalmente diferente na integração de LLMs em sistemas de software. Em vez de tratar o modelo de linguagem como um oráculo de acesso direto — no qual o prompt entra e a resposta sai sem mediação estruturada — o middleware interpõe um pipeline de raciocínio epistemológico que torna o processo transparente, auditável e formalmente fundamentado.

A combinação de extração conceitual aristotélica, classificação kantiana, filtragem científica por Hempel e Popper, avaliação paraconsistente baseada em Newton da Costa e verificação por equivalência russelliana não é uma escolha estética — cada elemento resolve um problema técnico concreto identificado nos LLMs puramente neurais: ambiguidade semântica não resolvida, ausência de hierarquia epistêmica, hipóteses irrelevantes ou não falsificáveis, explosões lógicas ante contradições locais e opacidade do raciocínio intermediário.

O sistema encontra aplicação natural em domínios onde a confiabilidade e a rastreabilidade das respostas de IA são juridicamente ou metodologicamente relevantes: medicina, direito, ciência, educação e qualquer contexto em que "o modelo disse isso" não seja suficiente sem saber por quê, com que grau de certeza e a partir de quais evidências.

Licenciamento:

O uso da IA Doninha é permitido somente para usuários privados. Para uso comercial ou governamental, contate o desenvolvedor pelo e-mail danielfonseca420@proton.me para informações sobre licenciamento de uso mediante pagamento de royalties. A tecnologia IA Doninha encontra-se em processo de registro de patente no INPI (Brasil).

Daniel Barros Fonseca

Inventor e Arquiteto da IA Doninha

Fundamentação teórica: "Uma verdadeira Epistemologia para a Inteligência Artificial"